SATI Campus Complaint Management System: PRD

Code Crack 3.0 · Second Year · Samrat Ashoka Technological Institute · Base project: NeveshMJ/vit (GRIEVEX, MIT licence)

0. How to use this document

This is the single source of truth for the team and for the AI agents (Antigravity) working on it. Backend and logic are built by Dev; frontend design and UI by a teammate. Section 8 (API contract) is the interface between the two sides. If this PRD and the code disagree, the PRD wins. Do not add features that are not listed here.

Priority tags: **P0** = mandatory in the problem statement, **P1** = our extras, **P2** = stretch, only after P0 and P1 are done.

1. Problem and goal

Students report campus issues (Wi-Fi, classrooms, hostels, laboratories). Administrators need one place to track them and see which unresolved complaints need attention first.

Goal: a working complaint dashboard with a complaint form, searchable and filterable list, editable status, summary counts and a priority-ordered queue, where every priority score is explainable.

Pitch line: A transparent, SLA-aware complaint dashboard for SATI: every priority score is explainable, duplicate reports merge instead of flooding the queue, and every status change is auditable.

Constraints from the problem statement: 3 to 3.5 hour build window. No real authentication, email or SMS. Resolved complaints must never appear in the priority queue. Handle empty lists and invalid form input. Persist records, or document that data is session-only.

2. Judging rubric and where we win it

Criterion	Pts	Covered by
Complaint workflow and validation	30	M1, M2, M5, M11, E3, E4
Search, filtering and summary	20	M3, M4, M6
Priority logic and explanation	25	M8, M9, M10, M12, E1, E2, E5
Data handling	15	M7, E6
Usability	10	Section 11

3. Mandatory requirements (P0)

ID	Requirement	Acceptance criteria
M1	Complaint form	Category (fixed list) and description are required. Optional location/room field. Blank, whitespace-only or under 10 character descriptions are rejected with inline errors. Success message shows the new ID.
M2	Unique ID	Format `C05`, `C06`, ... Next ID = highest existing number + 1, zero-padded to 2 digits. Never derived from array length, never reused.
M3	Display complaints	List shows ID, category, description, status, age and urgency. Handles zero records.
M4	Search and filter	Filter by category and by status, plus free-text search on description and ID. Filters combine (AND). Clear-filters button. Empty result state.
M5	Status transitions	Raised to In Progress to Resolved. Forward only, one step at a time, Resolved is terminal. Server returns 400 for any other transition. UI only offers the valid next action.

ID	Requirement	Acceptance criteria
M6	Summary counts	Total, Pending, Resolved. Pending = Raised + In Progress. Total = Pending + Resolved. Updates immediately after any change. Also show the Raised / In Progress split.
M7	Persistence	Records survive refresh and server restart (section 9). Documented in README.
M8	Urgency score	Every unresolved complaint gets a score from category and age (section 6.5), computed live from timestamps, in one module with unit tests.
M9	Top 3 priority queue	Three highest-priority unresolved complaints with rank, score, breakdown and the tie-break rules printed on the page. Shows fewer than 3 if fewer exist.
M10	Resolved excluded	Resolved complaints never appear in the queue. Resolving a complaint removes it from the queue immediately. Covered by a unit test.
M11	Validation and empty states	Server-side validation returns field errors. UI shows inline messages, never alert(). Empty states for: no complaints, no search results, no unresolved complaints (All clear).
M12	Documentation	README documents the scoring formula, tie-break order, definition of pending, persistence method, and credits vit.

4. Our extras: the unique solution (P1)

ID	Feature	Behaviour
E1	Score breakdown and Why is this #1	Every queue item shows Category +2, Age +2, Reports +0 = 4. A popover explains which tie-break step separated it from the next item.

ID	Feature	Behaviour
E2	SLA overdue badge	Each category has a target resolution time. Unresolved complaints past target show Overdue by Xh. Display only, does not change the score.
E3	Duplicate detection and Me too	On submit, if a similar unresolved complaint exists in the same category, warn and offer Add +1 to C01 or Submit anyway. Each extra report adds to the score (max +3).
E4	Status audit trail	Per-complaint timeline of every transition with timestamps.
E5	Score simulator	Panel to change category weights and the age divisor and watch the queue re-rank live. Browser-only, never changes saved data or the official score.
E6	Data tools	Export JSON, import JSON (accepts organizer records, including an hoursAgo field), reset to sample data.

P2 stretch: reopen Resolved to Raised with a mandatory reason (a documented exception to forward-only), charts by category, CSV export, dark mode.

5. Out of scope (do not build or re-add from vit)

Login, signup, roles, OTP, email or SMS, photo or camera capture, GPS and maps, weather, any AI, LLM or vision API, fake or deepfake detection, ratings and feedback, provider assignment, Tamil Nadu districts.

6. Domain rules

6.1 Categories and weights

Category	Weight	SLA target (hours, editable config)
Laboratory	3	48

Category	Weight	SLA target (hours, editable config)
Wi-Fi	2	24
Hostel	2	12
Classroom	1	72

6.2 Statuses

`Raised` then `In Progress` then `Resolved`. No Rejected state.

6.3 Definitions

- **Pending** = Raised + In Progress. **Total** = Pending + Resolved.
- **hoursPending** = (now minus submittedAt) in hours, for any unresolved complaint. Moving to In Progress does not reset age.
- Time is always computed from stored absolute timestamps, never stored as a number of hours.

6.4 ID generation

`C` + number, where number = max existing number + 1, padded to at least 2 digits.

6.5 Urgency score (official rule)

```
base    = CATEGORY_WEIGHT[category]
age     = floor(hoursPending / 24)
reports = min(supportCount, 3)     // extra Me too reports, default 0
score   = base + age + reports
```

With no extra reports this equals the baseline formula in the problem statement (category weight + floor(hours pending / 24)), so results on the provided data are identical to the baseline.

6.6 Tie-break order (applied in this order)

1. Higher score
2. More hours pending (older first)
3. Higher category weight

4. Raised before In Progress (nobody has touched it yet)
5. Lower ID number (deterministic)

6.7 Urgency bands (display only)

Score 6 or more = High, 4 to 5 = Medium, 3 or less = Low. Always show the text label next to the colour.

6.8 Duplicate detection (E3)

Lowercase, strip punctuation, remove stopwords, split into tokens. Compare against unresolved complaints of the same category using Jaccard similarity. A match is similarity of 0.4 or higher. Return the best match only.

7. Data model

Field	Type	Notes
id	string	`C01` style
category	enum	Wi-Fi, Hostel, Classroom, Laboratory
description	string	trimmed, 10 to 500 characters
location	string, optional	max 80 characters
status	enum	Raised, In Progress, Resolved
submittedAt	ISO datetime	absolute timestamp
supportCount	integer	extra Me too reports, default 0
statusHistory	array	items of from, to, at

Seed data (submittedAt = now minus hoursAgo at first run). Descriptions are placeholders until organizers supply real ones.

ID	Category	hoursAgo	Status	Placeholder description
C01	Wi-Fi	48	Raised	Wi-Fi signal keeps dropping near the library
C02	Hostel	8	In Progress	Water supply interrupted in the hostel block

ID	Category	hoursAgo	Status	Placeholder description
C03	Classroom	72	Raised	Projector not working in the lecture room
C04	Laboratory	24	Resolved	Lab computers failing to boot

8. API contract (backend and frontend meet here)

Base path `/api`. JSON only. Errors: HTTP 400 with an `errors` object keyed by field name, plus a `message`. Unknown ID: 404.

Method	Path	Purpose
GET	/config	Weights, SLA targets, tie-break text, urgency bands, categories, statuses. The frontend reads rules from here and never hardcodes them.
GET	/complaints	Query params: category, status, q (search), sort (priority, newest, oldest). Returns an array of complaint views.
POST	/complaints	Body: category, description, location (optional). Validates, generates ID, returns the created complaint view.
POST	/complaints/check-duplicate	Body: category, description. Returns the best similar unresolved complaint or null.
GET	/complaints/:id	One complaint view including statusHistory.
PATCH	/complaints/:id/status	Body: to. Only the next valid status is accepted.
POST	/complaints/:id/support	Adds one Me too report. Rejected for resolved complaints.
GET	/stats	total, pending, resolved, raised, inProgress, byCategory.
GET	/priority-queue	Query param: limit (default 3). Unresolved only, ranked with tie-breaks. Each item includes rank and a tieBreakNote.
GET	/export	Full data as JSON.

Method	Path	Purpose
POST	/import	Accepts a JSON array. Accepts submittedAt or hoursAgo. Returns imported and skipped counts with reasons.
POST	/reset	Restores the sample data.

Complaint view = all stored fields plus computed fields:

Computed field	Meaning
hoursPending	number, one decimal
score	object with base, age, reports, total (null when resolved)
urgencyBand	High, Medium, Low or null
slaTargetHours	number
overdueByHours	number, 0 if not overdue or resolved
nextStatus	the only valid next status, or null

9. Persistence

- Default: a JSON file on the backend (`backend/data/complaints.json`), behind a small repository module so storage can be swapped later.
- Writes are atomic (write a temp file, then rename).
- On first run, or if the file is missing, seed the sample data.
- If the file is corrupted, back it up as `complaints.corrupt.json`, reseed, and surface a warning in the UI.
- No database service, no API keys. The app must start with `npm install` and one start command per side.

10. Codebase plan from vit (GRIEVEX)

Delete: `utils/vision.js`, `utils/gemini.js`, `utils/mailer.js`, `routes/auth.js`, `routes/provider.js`, `middleware/auth.js`, `models/User.js`, MongoDB and Mongoose setup, `Login`, `Signup`, `VerifyOTP`, `ProviderDashboard`, `ManageProviders`, `ComplaintMap`, `WeatherReport`, webcam and photo code, the 37 district data, rating and fake-detection logic, all API keys and `.env` entries. Do not reuse the demo credentials published in vit's README.

Keep and adapt: Express app structure, React app and router, `api.js` Axios instance (remove the JWT interceptor), `Sidebar.js`, the CSS variable design system in `App.css`, the status timeline idea, the KPI card layout from `ManagementDashboard.js`, the filter table idea from `ManagementComplaints.js`.

Write new: `utils/priority.js` (score, rank, tie-breaks), `utils/duplicates.js`, `store/complaintStore.js` (JSON repository), `routes/complaints.js` (rewritten), `seed.js`, unit tests, README.

11. Frontend specification

11.1 Principles

- Single-page React app, four screens, left sidebar (collapsible on mobile). No login.
- SATI name and a logo placeholder in the header. Teammate chooses the visual identity.
- Keep vit's approach of CSS custom properties in `App.css`. Starting palette: primary `#1e3a8a` to `#2563eb`, secondary `#0ea5e9`, typeface Inter. Free to restyle, but every colour must come from variables.
- All rules (weights, SLA, tie-break text, bands) are read from `GET /config`. Nothing is hardcoded in components.
- The frontend can be built before the backend is ready by using a mock file `docs/mock-data.json` that follows the section 8 shapes.
- Responsive from 360px width up: table on desktop becomes cards on mobile.
- Use text labels with colour, never colour alone.

11.2 Screens

A. Dashboard

- Three summary cards: Total, Pending, Resolved, with a small Raised / In Progress split under Pending.
- Priority Queue panel: top 3 cards. Each shows rank badge, ID, category chip, description, status chip, hours pending, large score number, breakdown chips (Category +2, Age +2, Reports +0), overdue badge, a Why is this #1 popover, and an Advance status button.
- How priority works (collapsible): the formula, the weights table, the numbered tie-break order, all read from config.
- Score simulator (E5) in a side drawer.
- All-clear empty state when nothing is unresolved.

B. Complaints

- Filter bar: search box, category select, status select, sort select, Clear filters. Text line: Showing X of Y.
- Row or card: ID, category chip, description (truncated), location, status chip, age, urgency chip (a dash for resolved), advance action.
- Clicking a row opens a detail drawer: full description, score breakdown, SLA badge, status timeline (E4), Me too button (E3).

C. Raise Complaint

- Category dropdown (4 options), description textarea with live character counter (10 to 500), optional location.
- Inline field errors under each input. Submit shows a spinner, then a success state with the new ID and a link to the complaint.
- Duplicate warning panel (E3) with Add +1 to Cxx and Submit anyway.

D. Tools

- Export JSON button, Import JSON file picker with a result summary (imported N, skipped M with reasons), Reset to sample data with a confirmation dialog.

11.3 States every screen must handle

Loading (skeletons), error with a Retry button, empty list, no search results, and the storage corruption warning banner.

11.4 Component list

SummaryCard, PriorityCard, ScoreBreakdown, WhyPopover, CategoryChip, StatusChip, UrgencyChip, SlaBadge, FilterBar, ComplaintTable and ComplaintCard, ComplaintDrawer, StatusTimeline, ComplaintForm, DuplicateWarning, Toast, EmptyState, ConfirmDialog, SimulatorPanel, RulesPanel.

11.5 Visual conventions

Item	Convention
Status	Raised = amber, In Progress = blue, Resolved = green
Urgency	High = red, Medium = orange, Low = grey, each with its text label
Category	A distinct icon plus a neutral chip colour per category
Numbers	Score is the largest figure on a queue card

11.6 Accessibility

Visible labels on all inputs, keyboard-reachable actions and drawers, focus ring, aria-live region for toasts and validation errors, sufficient contrast.

12. Acceptance tests (inject a fixed now so results are stable)

1. On seed data: scores are C01 = 4, C02 = 2, C03 = 4, C04 not scored.
2. Queue order on seed data: C03, then C01 (older, 72h vs 48h), then C02. C04 never appears.
3. Hours pending 23.9 gives age 0, and 24.0 gives age 1.
4. Resolving C03 removes it from the queue immediately and Resolved count goes up by 1, Pending down by 1.
5. PATCH Raised to Resolved returns 400. PATCH Resolved to anything returns 400.
6. Description of only spaces is rejected. Description of 9 characters is rejected. Missing category is rejected.

7. After creating a complaint on seed data the new ID is C05. After deleting the file and restarting, seed data returns.
8. Two complaints with equal score and equal hours are ordered by category weight, then status, then ID.
9. Fewer than 3 unresolved complaints shows only those. Zero unresolved shows All clear.
10. A Me too report raises the score by 1 up to a maximum of +3.
11. Imported record with hoursAgo = 30 gets submittedAt = now minus 30 hours.
12. Counts: Total always equals Pending + Resolved.

13. Work split and timeline (about 3.5 hours)

Time	Backend (Dev, Antigravity)	Frontend (teammate)
0:00 to 0:40	Strip vit, boot with no keys, JSON store, seed	Design tokens, layout shell, mock data
0:40 to 1:20	Model, validation, ID, routes	Raise Complaint form and validation UI
1:20 to 2:00	Status transitions, filters, stats	Complaints list, filters, drawer
2:00 to 2:40	priority.js, queue endpoint, tests	Dashboard, priority cards, rules panel
2:40 to 3:10	Duplicates, import, export, reset	Extras UI: simulator, timeline, tools, empty states
3:10 to 3:30	README, bug fixes	Responsive pass, demo polish

Integration points: agree the section 8 shapes first. Switch the frontend from mock data to the live API as soon as `/complaints` and `/stats` work.

14. Demo and submission checklist

- App starts from a clean clone with no keys and no database service.

- README contains the scoring formula, tie-break order, pending definition, persistence note, extras list, and credit to NeveshMJ/vit (MIT).
- Demo order: submit a complaint with a validation error, submit a valid one, show the duplicate warning, advance a status, show counts update, show the top 3 with breakdown, resolve the top item and show it leave the queue, show the simulator, export and reset.
- Confirm the hackathon rules allow building on an existing open-source codebase, and declare it in the submission.

15. Rules for AI agents working on this repo

1. Follow this PRD. If a request conflicts with it, ask the human before changing scope.
2. Never add authentication, email, external APIs or API keys.
3. All scoring and ranking lives in `priority.js` only. Frontend never recomputes the official score.
4. Never hardcode complaint counts, rules or seed ages outside `seed.js` and config.
5. Every endpoint validates input server-side and returns the section 8 error shape.
6. Write or update unit tests for any change to scoring, tie-breaks, ID generation or transitions, and run them before finishing.
7. Do not use alert(), localStorage as the source of truth, or random sorting. Ties must follow section 6.6.
8. Keep changes small and match the existing code style. Do not delete files outside the section 10 list.